

Reviewer Pack — Real Stable Polynomial Root Count Separation for Monotone DN...

Entry 63ae2b41cb6d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 21:12:39 UTC

Real Stable Polynomial Root Count Separation for Monotone DNF

Entry ID: 63ae2b41cb6d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 21:12:39 UTC

1. Conjecture

Field A (mathematical branch): Real Stable Polynomials **Field B** (complexity object): Monotone DNF Formulas for k-CLIQUE

Statement:

Let f be a monotone DNF formula for the k-CLIQUE problem on n -vertex graphs. Let $P_f(x)$ be the real stable polynomial whose coefficients encode the inclusion-exclusion of f 's clauses. Then, the number of real roots of $P_f(x)$ is $\Omega(n)$, whereas for general monotone DNF formulas of size $\text{poly}(n)$, it is $O(\log n)$.

Rationale (proposer's reasoning):

Real stable polynomials encode positivity conditions on their roots, which may mirror the combinatorial rigidity of k-CLIQUE's clause structure. The root count acts as a proxy for the formula's 'algebraic complexity', potentially exposing inherent limitations in representing k-CLIQUE with small DNFs.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a6926cdd45a35b67

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n, k):
        if n < k:
            return None
        vertices = list(range(n))
        clique = random.sample(vertices, k)
        for i in range(k):
            for j in range(i + 1, k):
                if (clique[i], clique[j]) not in edges and (clique[j], clique[i]) not in edges:
                    edges[(clique[i], clique[j])] = True
        return clique

    def real_stable_polynomial(clause_set):
        n = len(clause_set)
        poly = [1] * (n + 1)
        for clause in clause_set:
            monomial = 1
            for var in clause:
                monomial *= (x - var) / (x + var)
            poly = polynomial_multiplication(poly, monomial)
        return poly

    def polynomial_multiplication(p1, p2):
        result = [0] * (len(p1) + len(p2) - 1)
        for i in range(len(p1)):
            for j in range(len(p2)):
                result[i + j] += p1[i] * p2[j]
        return result

    def sturm_sequence(poly):
        seq = [poly]
        while True:
```

```

        derivative = [i * coeff for i, coeff in enumerate(seq[-1][1:], start=1)]
        if not derivative or all(coeff == 0 for coeff in derivative):
            break
        seq.append([-coeff for coeff in derivative])
    return seq

def count_real_roots(poly):
    seq = sturm_sequence(poly)
    sign_changes_pos = len([i for i in range(len(seq) - 1) if seq[i][0] * seq[i + 1][0] < 0])
    sign_changes_neg = len([i for i in range(len(seq) - 1) if seq[i][-1] * seq[i + 1][-1] < 0])
    return sign_changes_pos - sign_changes_neg

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    k = random.randint(2, min(n // 2, 5))
    clique_poly = real_stable_polynomial(generate_k_clique(n, k))
    if clique_poly is None:
        continue
    root_count = count_real_roots(clique_poly)
    results.append((n, k, root_count))

if not results:
    return {
        "metric_name": "Real Roots Count",
        "metric_value": 0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "k_clique_generation_failed"
    }

avg_root_count = sum(root_count for _, _, root_count in results) / len(results)
if avg_root_count < n_values[-1] / 10:
    return {
        "metric_name": "Real Roots Count",
        "metric_value": avg_root_count,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": f"Average root count {avg_root_count} < n/10 for some k-CLIQUE instances"
    }

return {
    "metric_name": "Real Roots Count",
    "metric_value": avg_root_count,
    "instances_tested": len(results),
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:

```

```

trial_result = run_trial(seed)
print(f"TRIAL: {trial_result}")
results.append(trial_result)

avg_metric_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={avg_metric_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={avg_metric_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='average_root_count < n/10' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11971f3e.py", line 108, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11971f3e.py", line 69, in run_trial
    clique_poly = real_stable_polynomial(generate_k_clique(n, k))
                                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_11971f3e.py", line 28, in generate_k_clique
    if (clique[i], clique[j]) not in edges and (clique[j], clique[i]) not in edges:
                                   ^^^^^
NameError: name 'edges' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to NameError before producing results, preventing evaluation of conjecture support/falsification. | next: Fix the NameError in generate_k_clique by properly initializing edges, then rerun tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	57120
2	propose	ollama_remote	qwen3:8b	0	0	139624
3	propose	ollama_remote	qwen3:8b	0	0	105082
4	propose	ollama_remote	qwen3:8b	0	0	69751
5	preregistration	ollama_remote	qwen3:8b	0	0	32431
6	novelty	ollama_remote	qwen3:8b	0	0	28012
7	novelty	ollama_remote	qwen3:8b	0	0	22757
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20861
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10277
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13659
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12645
12	judge	ollama_remote	qwen3:8b	0	0	22873

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 535091 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/63ae2b41cb6d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/63ae2b41cb6d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/63ae2b41cb6d.tar.gz` (if generated)